

BBC - auto categorize the news into various categories which will help in recommending the right news articles to it users at a later stage.

Importing Libraries

```
In [19]: import pandas as pd
import numpy as np
from IPython.display import Image
from tqdm import tqdm
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score
import re
import matplotlib.pyplot as plt
```

```
In [ ]:
```

Reading the dataset

```
In [20]: !gdown 1P0mzYEQZgZVqJ03F_cyNcd4QmoNE_mCv
```

Downloading...

From: https://drive.google.com/uc?id=1P0mzYEQZgZVqJ03F_cyNcd4QmoNE_mCv

To: C:\Users\Home\bbc-news-data.csv

0%		0.00/5.08M [00:00<?, ?B/s]
10% #		524k/5.08M [00:00<00:01, 2.86MB/s]
31% ###		1.57M/5.08M [00:00<00:00, 5.42MB/s]
52% #####1		2.62M/5.08M [00:00<00:00, 6.43MB/s]
72% #####2		3.67M/5.08M [00:00<00:00, 5.96MB/s]
93% #####2		4.72M/5.08M [00:00<00:00, 6.28MB/s]
100% #####		5.08M/5.08M [00:00<00:00, 6.12MB/s]

```
In [21]: df = pd.read_csv("bbc-news-data.csv", sep="\t")
df
```

Out[21]:

	category	filename	title	content
0	business	001.txt	Ad sales boost Time Warner profit	Quarterly profits at US media giant TimeWarne...
1	business	002.txt	Dollar gains on Greenspan speech	The dollar has hit its highest level against ...
2	business	003.txt	Yukos unit buyer faces loan claim	The owners of embattled Russian oil giant Yuk...
3	business	004.txt	High fuel prices hit BA's profits	British Airways has blamed high fuel prices f...
4	business	005.txt	Pernod takeover talk lifts Domecq	Shares in UK drinks and food firm Allied Dome...
...	...	...	...	...
2220	tech	397.txt	BT program to beat dialler scams	BT is introducing two initiatives to help bea...
2221	tech	398.txt	Spam e-mails tempt net shoppers	Computer users across the world continue to i...
2222	tech	399.txt	Be careful how you code	A new European directive could put software w...
2223	tech	400.txt	US cyber security chief resigns	The man making sure US computer networks are ...
2224	tech	401.txt	Losing yourself in online gaming	Online role playing games are time-consuming,...

2225 rows × 4 columns

In []:

Dropping the empty sub-title

```
In [22]: df = df[((~df.title.isnull()) & (~df.content.isnull()))].reset_index(drop=True)
print(df.shape)

df

(2225, 4)
```

Out[22]:

	category	filename	title	content
0	business	001.txt	Ad sales boost Time Warner profit	Quarterly profits at US media giant TimeWarne...
1	business	002.txt	Dollar gains on Greenspan speech	The dollar has hit its highest level against ...
2	business	003.txt	Yukos unit buyer faces loan claim	The owners of embattled Russian oil giant Yuk...
3	business	004.txt	High fuel prices hit BA's profits	British Airways has blamed high fuel prices f...
4	business	005.txt	Pernod takeover talk lifts Domecq	Shares in UK drinks and food firm Allied Dome...
...	...	...	...	...
2220	tech	397.txt	BT program to beat dialler scams	BT is introducing two initiatives to help bea...
2221	tech	398.txt	Spam e-mails tempt net shoppers	Computer users across the world continue to i...
2222	tech	399.txt	Be careful how you code	A new European directive could put software w...
2223	tech	400.txt	US cyber security chief resigns	The man making sure US computer networks are ...
2224	tech	401.txt	Losing yourself in online gaming	Online role playing games are time-consuming,...

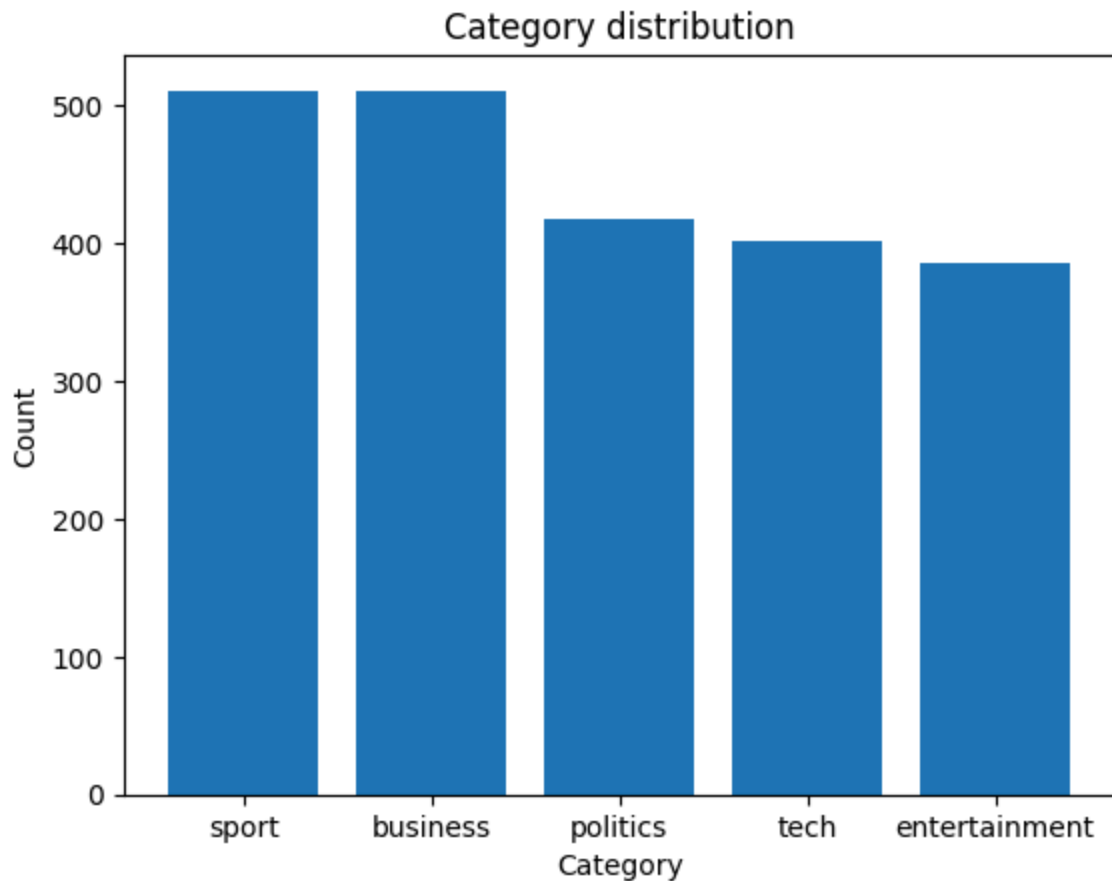
2225 rows × 4 columns

In []:

Visualization of category distribution

```
In [23]: plt.bar(df.category.value_counts().index, df.category.value_counts().values)
plt.title('Category distribution')
plt.xlabel("Category")
plt.ylabel("Count")
```

Out[23]: Text(0, 0.5, 'Count')



In []:

sample example of how the data

```
In [24]: ind=1807
print(f'Title: {df.title[ind]}')
print(f'Category: {df.category[ind]}')
```

Title: Federer joins all-time greats
Category: sport

In []:

Importing tensorflow and keras

```
In [38]: from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Dense, Embedding, Input, InputLayer, RNN, SimpleRNN
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.text import Tokenizer
import tensorflow as tf

import string

import nltk
```

```

nltk.download('stopwords')

from nltk.corpus import stopwords
stop_words = stopwords.words('english')

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
import seaborn as sns

```

```

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Home\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

In []:

In []:

Text cleaning

```

In [39]: def data_cleaning(text):

    # Lower the words in the sentence
    cleaned = text.lower()

    # Replace the full stop with a full stop and space
    cleaned = cleaned.replace(".", ". ")

    # Remove the stop words
    tokens = [word for word in cleaned.split() if not word in stop_words]

    # Remove the punctuations
    tokens = [tok.translate(str.maketrans(' ', ' ', string.punctuation)) for tok in tokens]

    # Joining the tokens back to form the sentence
    cleaned = " ".join(tokens)

    # Remove any extra spaces
    cleaned = cleaned.strip()

    return cleaned

```

```

In [40]: for index, data in tqdm(df.iterrows(), total=df.shape[0]):
    df.loc[index, 'title'] = data_cleaning(data['title'])

```

```

100%|████████████████████████████████████████████████████████████████████████████████| 2
225/2225 [00:00<00:00, 2856.38it/s]

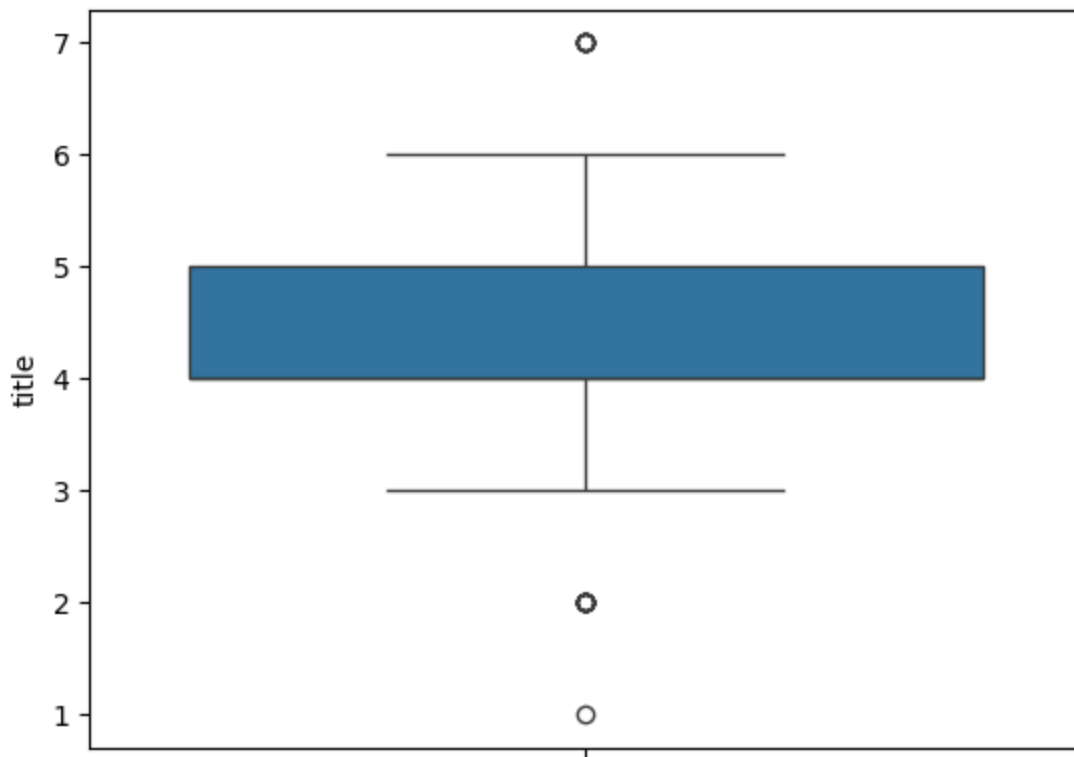
```

In []:

Fixing the sequence length

```
In [41]: sns.boxplot(df['title'].str.split(" ").str.len())
```

```
Out[41]: <Axes: ylabel='title'>
```



maximum number of words in the title is 7, we can set the sequence length to be the max words in the longest sentence.

```
In [42]: max_sentence_len = df['title'].str.split(" ").str.len().max()
total_classes = df.category.nunique()

print(f"Maximum sequence length: {max_sentence_len}")
print(f"Total classes: {total_classes}")
```

Maximum sequence length: 7

Total classes: 5

Splitting the data to train and test

```
In [43]: np.random.seed(100)
train_X, test_X, train_Y, test_Y = train_test_split(df['title'], #X
                                                    df['category'], #y
                                                    test_size=0.2,
                                                    random_state=100)

train_X = train_X.reset_index(drop=True)
test_X = test_X.reset_index(drop=True)
train_Y = train_Y.reset_index(drop=True)
test_Y = test_Y.reset_index(drop=True)
```

```
In [44]: train_X.shape, train_Y.shape, test_X.shape, test_Y.shape
```

```
Out[44]: ((1780,), (1780,), (445,), (445,))
```

In []:

One hot Encode the labels

```
In [45]: train_Y = pd.get_dummies(train_Y).values
         test_Y = pd.get_dummies(test_Y).values
```

```
In [46]: validation = test_Y.argmax(axis=1)
```

In []:

Tokenize the input text and pad them

```
In [47]: def tokenize_and_pad(inp_text, max_len, tok):

         text_seq = tok.texts_to_sequences(inp_text)
         text_seq = pad_sequences(text_seq, maxlen=max_len, padding='post')

         return text_seq

text_tok = Tokenizer()
text_tok.fit_on_texts(train_X)
train_text_X = tokenize_and_pad(inp_text=train_X, max_len=max_sentence_len, tok=text_tok)
test_text_X = tokenize_and_pad(inp_text=test_X, max_len=max_sentence_len, tok=text_tok)
vocab_size = len(text_tok.word_index)+1

print("Overall text vocab size", vocab_size)
```

Overall text vocab size 3339

In []:

Latent dimension and embedding dimension

Latent dimension: Dimension of the weight matrix U, V, W

Embedding dimension: Dimension of the word embeddings at the embedding layer

```
In [48]: latent_dim=50
         embedding_dim=100
```

In []:

Define the RNN model architecture

The embedding layer with 100 dimension

A single Vanilla RNN unit with 50 dimensions

A final output layer with 5 units(5 classes) with the softmax activation function

```
In [49]: seed=56
tf.random.set_seed(seed)
np.random.seed(seed)

model = Sequential()
model.add(Embedding(vocab_size, embedding_dim, trainable=True))
model.add(SimpleRNN(latent_dim, recurrent_dropout=0.2, return_sequences=False, activation='tanh'))
model.add(Dense(total_classes, activation='softmax'))
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	
embedding_2 (Embedding)	?	0
simple_rnn (SimpleRNN)	?	0
dense (Dense)	?	0



Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

In []:

Model training

Optimizer: Adam

Loss: Categorical cross-entropy since it is a multiclass classification problem

Early stopping: Used to stop training if validation accuracy does not improve while training to avoid overfitting

```
In [50]: import datetime

tf.random.set_seed(seed)
np.random.seed(seed)

model.compile(optimizer='Adam', loss='categorical_crossentropy',
              metrics=['acc'])

early_stopping = EarlyStopping(monitor='val_acc',
                               mode='max',
                               verbose=1,
```



```

patience=5)

log_dir = "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S")
tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir=log_dir, histogram_fr

model.fit(x=train_text_X, y=train_Y,
          validation_data=(test_text_X, test_Y),
          batch_size=64,
          epochs=10,
          callbacks=[early_stopping, tensorboard_callback])

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/10

28/28 ————— 5s 39ms/step - acc: 0.2618 - loss: 1.5857 - val_acc: 0.2629 - val_loss: 1.5592

Epoch 2/10

28/28 ————— 1s 17ms/step - acc: 0.5904 - loss: 1.3186 - val_acc: 0.5258 - val_loss: 1.2421

Epoch 3/10

28/28 ————— 1s 18ms/step - acc: 0.8236 - loss: 0.6679 - val_acc: 0.7236 - val_loss: 0.8145

Epoch 4/10

28/28 ————— 1s 18ms/step - acc: 0.9472 - loss: 0.2582 - val_acc: 0.7281 - val_loss: 0.8619

Epoch 5/10

28/28 ————— 1s 21ms/step - acc: 0.9697 - loss: 0.1449 - val_acc: 0.7461 - val_loss: 0.9179

Epoch 6/10

28/28 ————— 1s 20ms/step - acc: 0.9860 - loss: 0.0811 - val_acc: 0.7438 - val_loss: 1.0102

Epoch 7/10

28/28 ————— 1s 17ms/step - acc: 0.9888 - loss: 0.0644 - val_acc: 0.7169 - val_loss: 1.1214

Epoch 8/10

28/28 ————— 0s 17ms/step - acc: 0.9933 - loss: 0.0447 - val_acc: 0.7146 - val_loss: 1.2006

Epoch 9/10

28/28 ————— 1s 17ms/step - acc: 0.9955 - loss: 0.0321 - val_acc: 0.7146 - val_loss: 1.2491

Epoch 10/10

28/28 ————— 1s 18ms/step - acc: 0.9938 - loss: 0.0301 - val_acc: 0.6944 - val_loss: 1.3747

Epoch 10: early stopping

Out[50]: <keras.src.callbacks.history.History at 0x1bb72888590>

In [52]: %load_ext tensorboard
%tensorboard --logdir logs/fit --port 6007

The tensorboard extension is already loaded. To reload it, use:

%reload_ext tensorboard

Reusing TensorBoard on port 6007 (pid 8600), started 1 day, 15:36:19 ago. (Use '!kill 8600' to kill it.)

Index of C:\

Name	Size	Date Modified
\$Recycle.Bin/		11/21/24, 2:39:31 PM
\$WinREAgent/		4/2/26, 7:26:39 PM
Documents and Settings/		9/18/22, 9:42:23 AM
inetpub/		4/22/25, 9:14:05 PM
Intel/		11/21/24, 2:59:32 PM
PerfLogs/		12/7/19, 2:44:52 PM
Program Files/		10/27/25, 10:28:21 AM
Program Files (x86)/		7/1/26, 7:56:38 PM
ProgramData/		5/11/26, 9:10:16 PM
Recovery/		11/21/24, 2:39:39 PM
System Volume Information/		9/17/22, 11:52:20 PM
Users/		9/17/22, 11:51:58 PM
Windows/		7/1/26, 7:56:42 PM
\$WINRE_BACKUP_PARTITION.MARKER	0 B	4/2/26, 7:26:27 PM
DumpStack.log	8.0 kB	11/20/25, 8:05:28 PM
DumpStack.log.tmp	8.0 kB	6/29/26, 10:24:38 AM
hiberfil.sys	2.4 GB	7/8/26, 9:03:34 AM
logUploaderSettings_temp.ini	108 B	8/4/25, 5:59:10 PM
logUploaderSettings.ini	108 B	8/4/25, 5:59:10 PM
pagefile.sys	12.0 GB	7/6/26, 5:55:57 PM
swapfile.sys	272 MB	6/29/26, 11:16:58 AM



In []:

Save the trained model

In [53]:

```
model.save("BCC_classifier.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

In []:

Load the saved model

In [54]: `model = tf.keras.models.load_model("BCC_classifier.h5")`

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

In []:

Make predictions on the test dataset

In [55]: `prediction = model.predict(test_text_X)
prediction = prediction.argmax(axis=1)
print(f"Accuracy: {accuracy_score(prediction, validation)}")`

14/14 ————— 1s 27ms/step
Accuracy: 0.6943820224719102

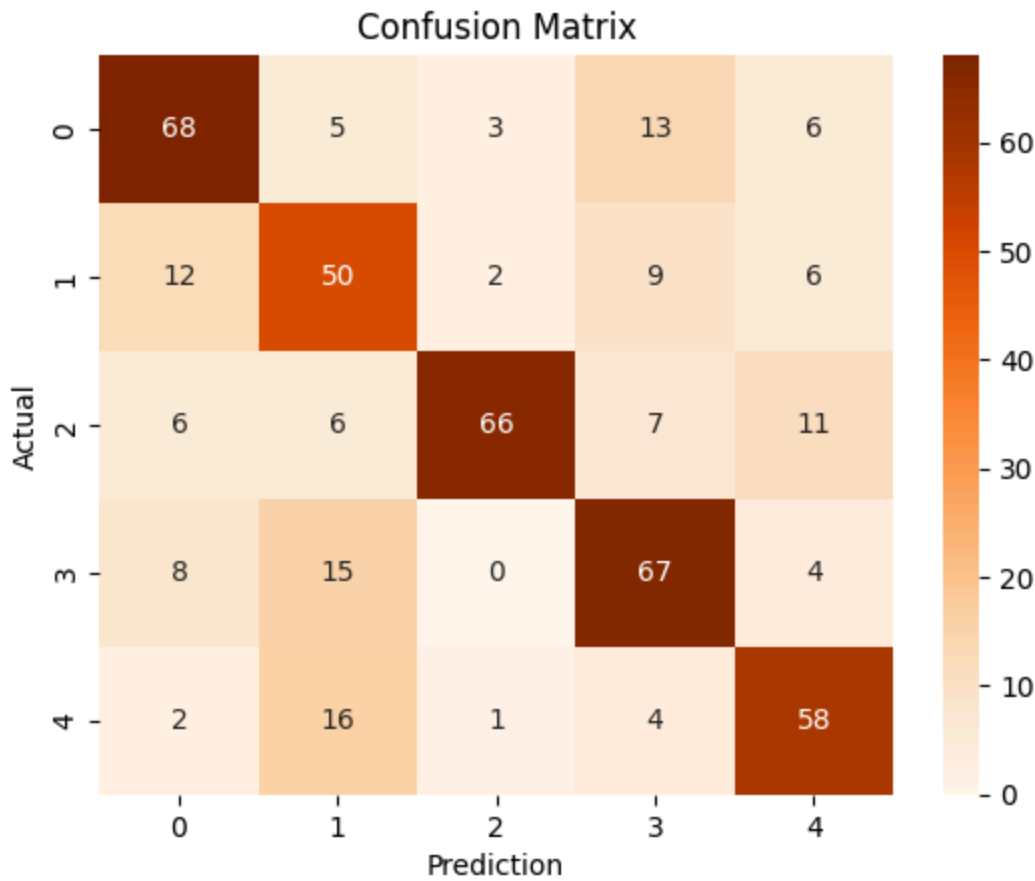
In []:

Confusion matrix of the prediction and actual

In [56]: `cm = confusion_matrix(validation, prediction)

print("")
plt.figure(figsize=(15,15))
sns.heatmap(cm, annot=True, cmap='Oranges')
plt.xlabel("Prediction")
plt.ylabel("Actual")
plt.title("Confusion Matrix")`

Out[56]: Text(0.5, 1.0, 'Confusion Matrix')



Concluion: - The diagonal cells represent correct predictions and are ideally large compared to other cells. - The off-diagonal cells represent misclassifications and help identify which classes the model confuses. - Class 0 is often mistaken for Classes 2 and 3. - Class 2 is confused with Classes 0 and 3. - Class 4 is most frequently confused with Class 3. - Model correctly classified 291 out of 445 validation samples, giving an overall accuracy of approximately 69.43%. - The matrix also shows that Class 3 is the most accurate, while Classes 0, 2, and 4 have more overlap and are more difficult for the model to distinguish.

```
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
In [ ]:
```